Education & Computing 8 (1993) 347–355
Elsevier

Portable AI Lab for teaching Artificial Intelligence

Michael Rosner and Fabio Baj

IDSIA, Corso Elvezia 36, 6900 Lugano, Switzerland

Abstract

Rosner, M. and F Baj, Portable AI Lab for teaching Artificial Intelligence, Education&Computing 8 (1993) 347–355.

The Portable AI Lab is a joint research project concerned with the design and implementation of an integrated environment to support teaching of Artificial Intelligence at University level. The system is composed of several modules implementing basic. AI techniques in a uniform way. The final section of the paper focuses primarily on the modules dealing with Automated Theorem Proving (ATP) and Natural Language Processing, and shows how relationships between the two might typically be explored within the system.

Keywords: artificial intelligence; logic; natural language processing; integrated environment.

1. Introduction

The transmission of AI expertise is severely hampered by the absence of high-quality materials suitable for education and applications-oriented research. Although there are many introductory textbooks on AI available, it is difficult to acquire expertise by simply reading them. Practical experience with the state of the art is generally only available to the privileged few who already know how to translate the unstructured mass of research reports, journal articles, manuscripts and programs into effective code.

The lack of easy-to-use and well-documented source code is a major obstacle to the development of AI applications in educational contexts. Most universities simply do not have the resources or the time to develop their own systems from literature descriptions.

Commercially available products such as expert system shells do not really offer a satisfactory alternative. Documentation is often lacking or incomplete. For reasons of confidentiality, the source code is usually unavailable or undocumented. From the perspective of education, such systems do not help understanding either the conceptual or implementation issues employed in their construction. From that of the developer, they are too inflexible and cannot be customized to the actual requirements. Last but not least, commercial shells are expensive, and often their quality is questionable.

2. The Portable AI Lab

The Portable AI Lab has been designed with these problems in mind. It is a computing environment containing a collection of state-of-the-art AI tools, examples, and documentation. It is aimed at those involved in teaching AI courses at university level or equivalent. The system is being developed under Swiss National Research Programme PNR 23 on AI and Robotics by IDSIA, Lugano in collaboration with the Institut für Informatik, University of Zürich and the Laboratoire d'Intelligence Artificielle at the Ecole Polytechnique Fédérale, Lausanne.

The acquisition of expertise in AI depends on extensive practical experience with a broad range

of AI problems that are typically *interdisciplinary* in that they involve the intersection of several subdomains.

An example may help to illustrate this. If we take the example of a fairly typical and general AI problem area, such as natural language interfaces to help systems, we are immediately faced with at least three such subdomains: one dealing with the parsing of text, one dealing with reasoning and inference mechanisms, and a third dealing with the representation of knowledge about the subject matter. It is not difficult to see how easy it might be for other domains to become involved. The help system, being intelligent, needs to maintain a model of what the user can be assumed to know. This immediately brings in the domains of user models and the representation of nested beliefs. Now add the reasonable requirement that the user, in invoking the help system, has a definite goal to achieve and that the system should help him to achieve it. To incorporate this requires many of the concepts from the theory of planning; and so on. Between each of these domains interfaces must by defined and implemented, and clearly, the character of the system as a whole will depend largely upon where the interfaces are drawn, what kind of representations pass between them.

To the experienced AI practitioner with some experience this comes as no surprise. For such a person, the solution of an AI problem is, almost by definition, bound to cross many traditional frontiers, and part of the expertise he or she can bring to bear concerns the setting up of machinery for the import and export of techniques from circumscribed domains.

Portable AI Lab has enough built-in functionality to allow the user to experience interdisciplinary problem solving without having to build all the supporting tools from scratch. It is intended to support teaching activities by providing enough material to encourage the exploration of domains and the relationships between them.

This functionality is provided though a number of *modules* covering different AI subdomains and associated techniques. The system currently consists of prototype versions of modules concerned with automated theorem proving, natural language processing, rule-based inference, truth maintenance systems, planning, learning, knowledge acquisition, genetic algorithms and neural networks. The system is still under development and further modules are planned. Each module comprises:

- a fully implemented computational theory over the domain in question;
- a representative set of running examples that demonstrate the computational theory;
- documentation (some online, some paper) including
  - a literature guide to standard bibliographic references;
  - a user guide explaining how to use the module in question (e.g. input language syntax, available options, etc.);
  - source code documentation containing details of the implementation. Eventually all modules will also include a programmer's guide to explain how to incorporate "contributed" code.

The whole system is implemented in Common Lisp + Common Windows, is distributed free of charge in source form, and is implemented in a uniform style. This, together with a *pool* facility for sharing data, is designed to encourage the user to design and implement interfaces between different modules.

Although a full discussion of the system is beyond the scope of this article, we attempt to convey how it might be used to investigate problems concerning the relationship between logic and language. The reader is referred to Allemang et al. [1] for an illustration of other interdisciplinary problems within the scope of the system. Section 3 discusses the theorem proving module in detail; Section 4 illustrates an approach to a complex problem, requiring in addition techniques from natural language processing.

3. The Automated Theorem Proving Module

The claim that logic is of fundamental importance to AI tends to generate a mixture of (mostly strong) reactions from AI practitioners (e.g. Hayes [2]). This is one of the reasons for including an automated theorem proving (ATP) module in the Portable AI Lab. Some other, less controversial reasons are that

- A considerable part of AI expertise consists of the ability to produce formal and declarative representations of the knowledge used to solve

problems. First-order logic is still one of the best developed and best studied formal languages for the representation of declarative knowledge (see Genesereth and Nilsson [3]).

- First-order logic can be used to formally characterize some interesting properties of many different application domains including mathematics, natural language analysis, program validation and synthesis, circuit design and verification, planning, and so on.
- ATPs are interesting as problem-solving devices in these domains, so that observing the behavior of an ATP at work can often suggest the way in which a formalization should be modified in order to achieve better results.
- ATPs can be used to show the relationship between logic and programming. Different programming paradigms such as logic programming and functional programming can be learned in the context of automated theorem proving.

However, there is number of problems with teaching the relation between theorem proving and AI at the graduate level. One concerns a gap that still remains between the acquisition of the basic concepts of logic (necessary for understanding how a theorem prover works) and the concrete ability to apply them. An example is the difference between knowing the definition of Robinson's resolution rule and being able to formalize a complex problem in first order logic in such a way that it can be solved by means of an automated system that uses that rule. A working ATP can be an excellent basis for experimental study of the relation between the basic concepts of symbolic logic and their practical use.

Another problem is that students sometimes fail to acquire a unified view of AI: and this may happen even in the subset of AI techniques dealing with logic. After having taken an AI course students might know something about Prolog programming, theorem proving, truth maintenance systems, natural language analysis, and so on. However, they will almost certainly know little about the potential connections between these parts of AI. This situation may cause students to become specialists in very restricted subfields. We have tried to address this problem in the Portable AI Lab by presenting the ATP in the context of other modules with which interfaces could in principle be defined. This provides a way to investigate the relationships between deductive reasoning and other aspects of intelligent behavior.

3.1. Overview of the ATP system

The ATP module is called LENprover and provides a significant range of well known techniques for theorem proving including resolution [4], paramodulation [5], term rewriting [6,7], equational reasoning [8], semantic attachment [3,9], question answering [10], and Prolog [11]. The system is a refutation-based theorem prover for first-order logic with equality working on the clausal representation of formulas as follows.

- The input language is full first-order logic, with quantifiers and connectives. The user may define infix or postfix functors. The conventional Prolog notation for clauses is also recognized. The system reads a theorem contained in a text file, making some preprocessing steps (for instance tranforming first-order formulas into clauses).
- Starting from the initial database of clauses, the main loop chooses a pair of clauses according to some criterion and applies an inference rule to them (from the ones selected by the user) to deduce a new clause which is added to the database. When a new clause is generated, some operations are performed to simplify the clause database (subsumption, simplification, demodulation, and semantic simplification), according to the options selected by the user.
- The general theorem proving algorithm used by the ATP module can be summarized as follows:

```
Prove (F)
  CL := clauses-of (F)
  Repeat
    choose clauses c1 and c2 from CL
    new-c  := an-inference-rule (c1, c2)
    new-c1 := simplify new-c with CL
    CL     := simplify CL with new-c1
  until Empty-Clause is found OR
     no more deductions are possible
  IF Empty-Clause is found THEN
    return ``contradiction''
  ELSE return ``consistent''
```

This generative process terminates when a contradiction (or an answering clause) has been found or no more deductions are possible.

The behaviour of this process is affected by a number of user-settable parameters as indicated in Fig. 1.

3.1.1. Syntax

Amongst the problems faced by the student of computational logic is the variety of syntactic conventions for representing clauses. The system therefore allows the selection of different output formats for clauses for establishing that

```
a(X, Y) :- b(X),
           c(Y).
a(x, y) | ~b(x) | ~c(y).
b(x) & c(x) => a(x, y).
```

are three equivalent ways of expressing the same clause.

3.1.2. Prolog, inference rules and strategy

Since a Prolog interpreter can be seen as a theorem prover which employs a particular inference rule (SLD resolution [12]), a particular search strategy, and a restricted class of formulas (Horn clauses), it is interesting to convey to the user how a general theorem prover can be adapted to operate exactly as a Prolog system. Of course, it is equally important to understand which class of problems can be succesfully confronted with Horn clauses rather than with full first-order logic. The implementation includes the cut symbol (!) and the traditional Prolog notation for lists, but not things like `assert`, `retract`, `clause`... which lie outside the domain of pure theorem proving activity.

The SLD resolution inference rule is simply one of the available inference rules, just as the linear-input depth-first search strategy is only one of the available strategies: their combination results in a Prolog system, but the user can play with every other combination. Running the prover in Prolog mode a user can see in detail the process of goal matching, and the graphic display of proof trees gives a clear idea of the Prolog search strategy as shown in Fig. 2. But the general theorem proving mechanism is open to more complex experiments: let us consider for example the classical left-recursive Prolog program:

```
ancestor(X, Y) :- ancestor(Z, Y),
                  parent(X, Z).
ancestor(X, Y) :- parent(X, Y)
```

Running this program with the Prolog strategy the user will observe an infinite growth of the database of deductions. What he can do at this point is select a different search strategy (for instance with a breadth-first component): in this way the prover will find the solution in few steps.

From this kind of experiment a student can learn that sometimes a logically correct description may be useless if the proof method used is not appropriate: in particular, he cannot write Prolog programs by simply describing *what* to perform: he also has to consider *how* the program will be executed. On the other hand the ATP module also provides a set of examples that are almost intractable with a general theorem proving system, but which became straightforward if the Prolog strategy is selected.

3.2. Semantic attachment

Since automated theorem proving consists in the manipulation of linguistic expressions according to some rules, no reference to the meaning of symbols is made during computations. Nevertheless these linguistic constructs generally denote

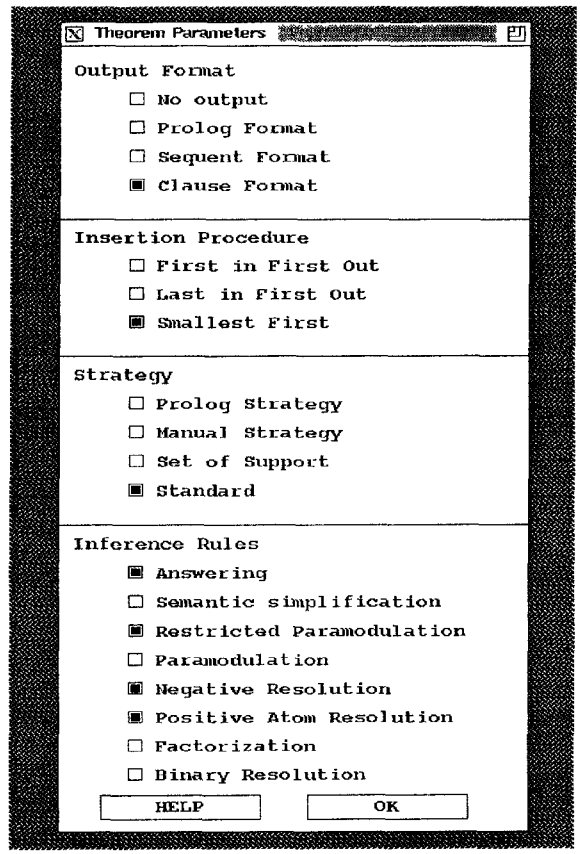

Fig. 1. User-settable parameters in the ATP module.

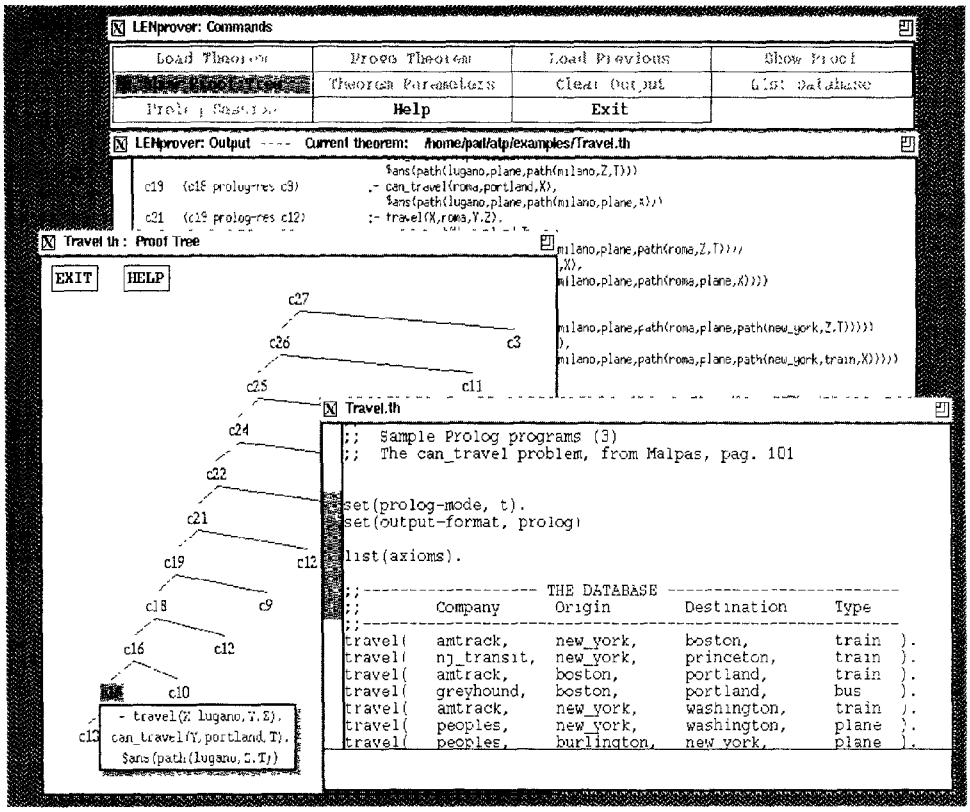

Fig. 2. A screen dump showing the ATP module at work on a Prolog theorem.

objects, functions and relations in a particular domain, and there are cases in which the intended interpretation of symbols is a standard and computable one [3]. For instance, if we are using the predicate symbol $>$ with the intended meaning of *being greater than*, then we might expect the prover to consider as true an expression like $5 > 3$, simply *evaluating* it instead of *demonstrating* it or adding to the axioms set facts like $(5 > 3)$ for all numbers. The evaluation of the expression $(5 > 3)$, can be done with a call to a computer program that compares numbers within the *finite* domain of the program. For example, consider the two formulas:

```
all x all y all z
 ((x+y) > (z-x)) =>
  P((z+y)) | Q(x, y, z).
~Q(1, 2, 3).
```

Normally the ATP module will deduce

```
((1+2) > (3-1)) => P((2+3))
```

but if the user selects the option of semantic attachment the prover will derive

```
P(5).
```

Notice that there is the need of a precise definition of the domain of the attached functions. For instance the prover has to call the program `plus` attached to $+$ only when both the arguments of $+$ are numerals, while expressions like $(x + 3)$ should not be evaluated. Semantic evaluation of formulas is a process of interpretation with respect to a partial model [9] which in some cases can be the underlying Lisp interpreter. The possibility of semantic simplification of complex linguistic expressions is of great importance in im-

proving the efficiency of theorem provers. It is also a powerful way to interface declarative knowledge with procedural knowledge in complex AI systems. For instance we can combine the symbolic reasoning features of a planner for the blocks world with a computable discription of certain physical characteristics such as stability or gravity.

3.2.1. Question answering

The use of a theorem prover as a Question Answering System (QAS) was first introduced by Green [10]. A simple theorem prover is itself a question answerer, but it is limited to stating whether a given fact is true or false. If we want to answer questions like "which individual has property P?" some extensions to the prover are needed. The usual way is to express such a question as a theorem of the form $\exists x P(x)$, and to trace the substitutions occurring to the variable x along the proof. These substitutions are recorded with a special marker (the *Answering Literal* [13]), and alternative answers can be computed. This mechanism is also used to provide answers when the prover is in Prolog mode. Here are examples of how the user may ask questions to the prover:

```
all x (fact(10, x) | $ans(x))
all x (fact(10, x) => ?(x))
 :- fact(10, X).
```

4. Natural language processing

In accordance with the general philosophy of the system, the natural language processing (NLP) module is a collection of well-known algorithms for processing text together with supporting documentation and examples.

NLP subsumes a wide range of tasks including analysis and synthesis of text, question answering, database interrogation, request processing and so on. The present version of the module deals only with the problem domain of sentence analysis, that is, the process of computing all and only structures assigned to it by some general specification of legitimate assignments.

One of the aims of the NLP module is to give the user some idea of the techniques available for providing such specifications. Of particular importance here is the distinction between *procedural* and *declarative* styles. The essential difference is that procedural specifications state *how* the computation is performed, whilst declarative ones state *what* the computation is supposed to do, leaving the steps, and the order in which they are executed, to be determined by more general principles.

The declarative approach is exemplified by a range of algorithms that have been developed from within both computer science and computational linguistics under the names of *tabular parsing* (see Cocke and Schwarz [14], Kasami [15], Younger [16] and Earley [17]) and *chart parsing* (see Kay [18], Kaplan [19] and Thompson [20]). The module that illustrates this approach, which is currently under development, attempts to show how the simplest Cocke–Kasami–Younger algorithm can be gradually extended to handle more sophisticated rules and run more efficiently.

One of the best illustrations of the procedural approach to parsing is that exemplified by Augmented Transition Networks (ATN), first described by Woods [21], in which a parsing device is described in a manner quite similar to a finite state machine. Nodes in the network correspond to parsing states, and arcs between nodes to steps in the parsing process.

A basic transition network grammar looks like a collection of finite state transition diagrams; each is a directed graph with labeled states and labeled arcs, a distinguished start state and a set of distinguished final states. This is illustrated in Fig. 3, which describes a part of a grammar for English. The start states are labelled with the name of the syntactic construct recognised by the subnetwork, and the final states are indicated with double circles.

The labels on arcs represent conditions that must fulfilled in order to effect a transition. A number of primitive constructs are available for describing such conditions, including "**cat** ⟨label⟩" (the next word in the input text is of category ⟨label⟩), and "**push** ⟨label⟩" (the subnetwork ⟨label⟩ is accepted from the current input position). An input sequence is said to be accepted by the network if, beginning in the start state, some path (sequence of arcs) can be followed which terminate with a final state.

ATNs differ from finite state automata in several ways, e.g. in allowing the possibility of recursion via push arcs (this gives the power of context

free grammars), and in permitting read–write operations on *registers* for the copying, deletion, and modification of sentence structure fragments, the formulation of tests that depend upon context, and finally, the creation, storage and retrieval of arbitrary structures. These differences are further described in Johnson [22].

ATNs have been extensively used for the description of grammars in a number of practical systems, particularly with respect to question answering and speech understanding. Both of these areas are tackled at once in the Lunar Project for answering questions about lunar rock samples (see Nash-Webber et al. [23]).

The ATN module currently implements a simple interpreter for simulating the behavior of a parser specified by a user-supplied grammar and lexicon. Debugging, tracing and stepping facilities are provided. A graphics mode is supplied whereby the grammar is displayed as a network. This is fully integrated so that active states can are shown when the interpreter is running. Several example grammars and lexicons are provided, as well as four autonomous demonstrations.

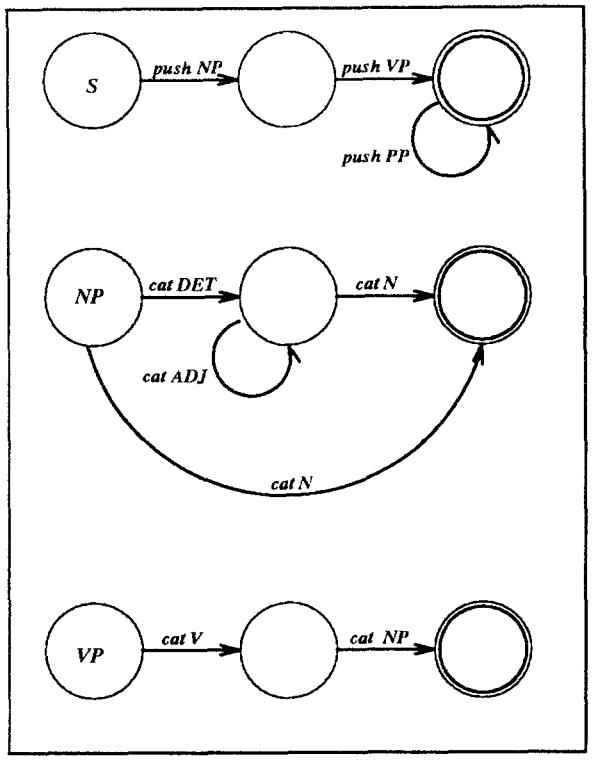

Fig. 3. A typical ATN grammar.

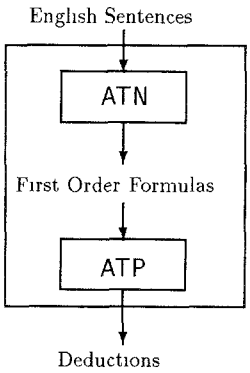

Fig. 4.

4.1. An interdisciplinary problem

Getting a computer to *understand* language is a good example of the kind of interdisciplinary problem that we suggested was characteristic of AI. It is generally agreed that the solution must somehow involve the interaction between processes that can handle the relation between linguistic *forms* of various kinds (e.g. morphological, lexical and syntactic), and *content* or meaning. However, there is no such general agreement about the mechanisms nor about the way to put them together.

The Portable AI Lab currently provides some mechanisms described above which, working together, can be used to exploring a small part of the problem: checking whether a set of sentences is contradictory, or deciding whether a sentence is logical consequence of a set of premises.

This is done designing an ATN which produces first-order formulas reflecting the semantics of the sentences it parses. One of the example grammars provides output compatible with theorem prover. These formulas are then piped to the theorem proving module which makes deductions, as shown in Fig. 4.

The screen dump of Fig. 5 shows the system during a session involving the ATN and ATP modules. The layout needs some explanation:

The *Main* window (top left corner of the screen) shows the two modules (ATP and ATN) which are currently running. Also visible are the other modules provided by PAIL, which are not

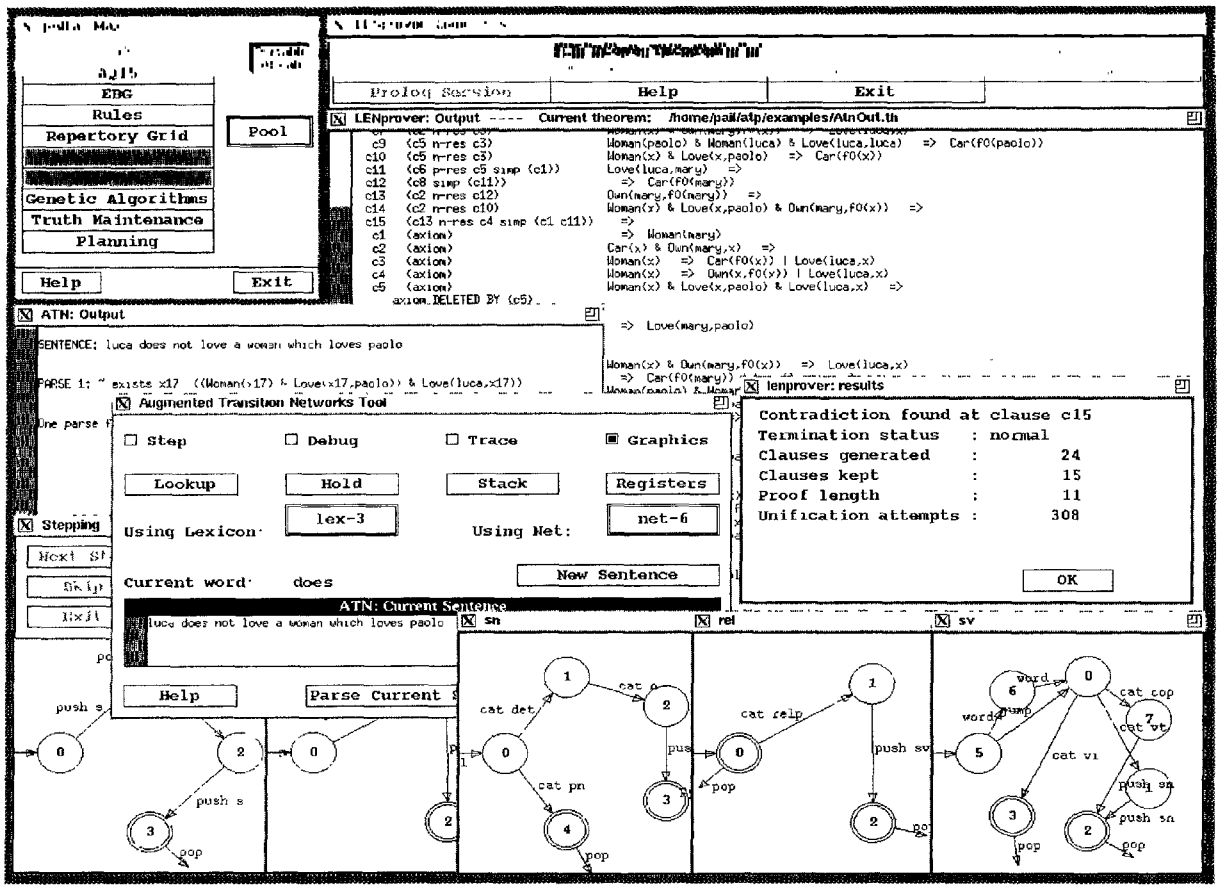

Fig. 5. The ATP and ATN modules working together.

active in this session. The windows of the theorem proving tool are on the top right of the screen. The *ATN Tool* window provides a means for the user to load networks and lexicons and to parse sentences with various levels of debug information. Executions of the parser may be stepped and graphically displayed as shown in the bottom of the screen. In the figure the ATN is parsing the sentence "Luca does not love a woman who loves Paolo".

The first-order formula produced by the network net-6 using lexicon lex-3 is displayed in the *ATN: output* window. As long as sentences are translated into formulas, these are also piped as input for the theorem proving module. The adoption of a uniform programming style makes the setting up of this kind of inter-process communication straightforward. The LENprover output window shows the ATP module at work proving that the formulas produced by the ATN module on the sentences:

1. Mary is a woman and she has no car
2. Luca does not love a woman who loves Paolo
3. Mary loves Paolo
4. Luca loves every woman who owns no car

lead to a contradiction, as displayed in the *LENprover: results* window.

References

[1] D. Allemang, R. Aiken, N. Almassy, T. Wehrle, and T. Rothenfluh, Teaching machine learning principles with the Portable AI Lab, in: *Proc. CALISCE, EPFL, Lausanne, Switzerland*, 1991.

[2] P.J. Hayes, In defence of logic, in: *Proc. Internat. Joint Conf. in Artificial Intelligence, Cambridge, MA* (1977) 559–565.
[3] M.R. Genesereth and N.J. Nilsson, *Logic Foundations of Artificial Intelligence* (Morgan Kaufmann, Los Altos, CA, 1987).
[4] J.A. Robinson, A machine-oriented logic based on the resolution principle, *J. ACM* **12** (1965) 23–41.
[5] L. Wos and G.A. Robinson. Paramodulation and set of support, in: *Symp. on Automatic Demonstration* (Springer, Berlin, 1970) 276–310.
[6] J. Hsiang, Rewrite method for theorem proving in first-order logic with equality, *J. Symbolic Comput.* **1** (1987) 133–151.
[7] J. Hsiang, Refutational theorem proving using term rewriting systems, *Artificial Intelligence* **25** (1985) 225–230.
[8] D.E. Knuth and P. Bendix, Simple word problems in universal algebras, in: *Proc. Conf. Computational Problems in Abstract Algebras* (Pergamon, Oxford, 1970) 263–298.
[9] R. Weyhrauch, Prolegomena to a theory of mechanized formal reasoning, *Artificial Intelligence* **13** (1980) 133–170.
[10] C. Green, Theorem-proving by resolution as a basis in question-answering systems, in: B. Meltzer and D. Michie, ed. *Machine Intelligence 4* (Edinburgh University Press, Edinburgh, 1969) 183–205.
[11] W. Clocksin and C. Mellish, *Programming in Prolog* (Springer, Berlin, 1981).
[12] J.W. Lloyd, *Foundations of Logic Programming* (Springer, New York, 2nd ext. edn. 1987).
[13] C.L. Chang and R.C.-T. Lee, *Symbolic logic and Mechanical Theorem Proving* (Academic Press, New York, 1971).
[14] J. Cocke and J.I. Schwartz, Programming languages and their compilers, Technical report, Courant Institute of Mathematical Sciences, New York University, 1970).
[15] T. Kasami, An efficient recognition and syntax analysis algorithms for context-free languages, Technical report, Air Force Cambridge Research Laboratory, Bedford, MA, 1965.
[16] D.H. Younger, Recognition and parsing of context-free languages in time *n* cubed, *Inform. and Control* **10** (1967) 189–208.
[17] J. Earley, An efficient context-free parsing algorithm, Technical report, PhD Thesis, Dept Computer Science, Carnegie Mellon University, 1968.
[18] M. Kay, Morphological and syntactic analysis, in: A. Zampolli, ed. *Linguistic Structures Processing* (North-Holland, Amsterdam, 1977) 131–234.
[19] R.M. Kaplan, A general syntactic processor, in: R. Rustin, ed. *Natural Language Processing* (Algorithmics Press, New York, 1973) 193–241.
[20] H.S. Thompson, Chart parsing and rule schemata in gpsg, Technical Report DAI Research Paper 165, Department of Artificial Intelligence, University of Edinburgh, 1981.
[21] W.A. Woods, Transition network grammers for natural language analysis, *Comm. ACM*, **10** (1970) 591–606.
[22] R.L. Johnson, Parsing with transition networks, in: M. King, ed. *Parsing Naural Language* (Academic Press, New York, 1983).
[23] B. Nash-Webber, W.A. Woods and R.M. Kaplan, The lunar sciences natural language information system: Final report, Technical Report BBN Report No. 2378, Bolt, Baranek and Newman Inc., Cambridge MA, 1972.

Michael Rosner received an MA degree in psychology and philosophy in 1973 from Oxford, and a diploma in computer science from Cambridge in 1974. Subsequently he studied AI at Essex, specialising in conversational logic. He spent a year as an IBM research fellow at Winchester, working on medical information systems. At ISSCO Geneva he worked on planning and dialogue systems, and the design and implementation of grammatical formalisms for machine translation. He headed the software team for the Eurotra machine translation project throughout 1986. His current interests include dialogue systems, declarative programming techniques, educational software and medical applications of artificial intelligence. He is currently co-director of the Dalle Molle Institute for AI, Lugano, and partly responsible for the institute's scientific collaboration in Switzerland and abroad. He currently holds the post of treasurer for the European Chapter of the Association for Computational Linguistics.

Fabio Baj graduated from the University of Milan in 1989 with a Laurea in Informatica, having specialised in in the fields of Logic Programming and Computational Logic. He joined the Istituto Dalle Molle IDSIA in Lugano in 1990 as a principle investigator for the Portable AI Lab project where his main contribution has been to design and implement a Lisp-based theorem proving system based on term rewriting and other techniques. His current interests also include the application of Artificial Intelligence techniques to Medicine.